Evidence-Backed Release Assurance for Autonomous Agent Deployments: Cryptographic Attestation, Fail-Closed Gates, and Tamper-Resilient Audit Infrastructure

Anonymous Submission

Abstract

Release pipelines for autonomous agents increasingly gate promotion on attested execution traces: signed records of what the agent did during its evaluation run. These attestations share an unaddressed weakness. The evidence collector is the only observer of the run, so an adversary who controls the agent and the collector need not forge or alter anything: it performs an action and does not record it. The resulting bundle is internally consistent, every signature verifies, and every integrity check passes, because nothing was tampered with.

We define *Witnessed Trace Completeness*, a completeness property stated relative to a declared set of *witnesses*: the tool endpoints, API gateways and sandbox brokers that actually serve agent actions. Each witness serves only sessions opened by an orchestrator-signed credential, issues receipts carrying a monotonic per-session sequence number and, at session end, signs a commitment to how many actions it served. A stateful, fail-closed release gate then *reconciles* the submitted trace set against those receipts and closings for the session it credentialed, rather than verifying a document. We prove that, under EUF-CMA security of the witness and orchestrator signatures, an adversary holding the agent, the collector and the collector's key cannot omit, fabricate, relabel or substitute a witnessed action; and we show that no verifier whose verdict is a function of one submitted document can achieve witnessed completeness or replay-freedom, which places record-format standards on one side of a structural boundary and stateful reconciliation on the other.

We implement the protocol in EVIASSURE, an evidence-bundle format that rides inside in-toto/SLSA envelopes with a domain-separated, depth-bound Merkle trace commitment, and evaluate it against executed in-toto/DSSE and TUF verifiers and re-implemented receipt and hash-chain designs. Against 7 omission attacks and an honest control, reconciliation detects 7/7, receipt-only verification 2/7, and per-issuer chaining 2/7; on the orthogonal tampering axis the gate blocks 16/17 vectors (95% CI [73.0, 99.0]) against 10 for a composed in-toto/DSSE + TUF + OPA pipeline, and we report the one it does not. We state the guarantee's boundary as plainly as the guarantee: it is exactly as wide as the deployed witness set, it says nothing about actions no witness mediates, and it approves a semantically misbehaving trace that is honestly recorded; held-out inspection over a 1,075-profile corpus recovers 50/50 structurally overt anomalies and 0/25 disguised ones.

1 Introduction

An autonomous agent is promoted to production (a new checkpoint, a widened tool scope, a new pipeline stage) on the strength of an evaluation run: a test suite, a red-team script, a canary task. Unlike a compiled artifact, whose behaviour is fixed by its code, the agent's behaviour during that run is a non-deterministic sequence of tool calls [12, 15, 33], and the security question at the gate is not *what the agent is* but *what it did*. Supply-chain provenance answers the first question well: in-toto, SLSA and Sigstore attest the origin and integrity of static artifacts [23, 30, 32], and that such attestation is needed is well established: taxonomies of documented supply-chain attacks enumerate hundreds of real incidents [18, 24], package-manager compromise is commonplace in interpreted-language ecosystems [9], and update systems have required dedicated defences since 2010 [7, 22, 26]. In-toto's runtime-trace and agent-decision predicates and the 2026 agent-provenance drafts extend the same record formats to execution traces [1, 2, 14, 16], and receiver-attested receipt schemes give each recorded action an attestation from a party other than the agent [10, 25].

All of these verify a *document*. The document is produced by the evidence collector, and the collector is the sole observer of the run. Consider an execution $A \to B \to C \to D \to E$ in which C is the action a reviewer would have blocked. An adversary who controls the agent and the collector (a compromised runner, a poisoned build step, an agent that has been prompt-injected into rewriting its own log [12]) does not forge C's absence. It performs C, and records $A \to B \to D \to E$. Every signature over that document verifies; the Merkle root,

hash chain or receipt set is consistent, because each was computed over what was recorded. A receipt proves an action happened; a set of receipts does not prove that no other action happened. We call this the *omission* attack, to distinguish it from tampering, and we show (Section 7.6) that executed in-toto/DSSE and TUF verifiers, per-issuer hash chains and per-action receipts each fail it, for a reason that Theorem 2 makes structural rather than incidental.

Our approach moves the observation out of the collector's trust domain. The components that already serve an agent's externally observable actions (the database proxy, the API gateway, the sandbox broker) become *witnesses*: each holds a signing key the adversary does not control, serves only sessions opened by an orchestrator-signed credential, keeps a monotonic counter per session, returns a signed receipt $(sid, \mathrm{seq}, H_{\mathrm{act}})$ with every action it serves, and at session end signs a *closing* (sid, n_j) committing to how many actions it served. The release gate holds state and *reconciles*: for the session it credentialed, every receipt authenticates under a registry-pinned witness key, every witness's receipts form exactly $1, \ldots, n_j$, every receipt binds to a submitted trace, and every trace that claims a mediated action carries a receipt. Omitting, fabricating, relabelling or substituting a witnessed action now requires forging a witness or orchestrator signature (Theorem 1). The guarantee is stated *relative to* the witness set: an action no witness mediates is outside it, and we say so, because a guarantee whose boundary is declared can be audited and one that assumes a perfect collector cannot.

We implement the protocol in EVIASSURE[1]: an evidence bundle that is an in-toto Statement carrying a domain-separated (RFC 6962), depth-bound Merkle commitment to the trace set, a signed trace count, the witness receipts and closings, and a nonce; and a fail-closed gate whose verdict is a deterministic function of the bundle, a pinned policy, a pinned key and witness registry, the credentialed session, and replay state. The bundle rides inside existing SLSA/in-toto tooling unmodified; what the gate adds is the state and the second trust domain a document cannot carry.

1.1 Contributions

1. **Witnessed Trace Completeness** (Definition 5): a completeness property relative to a declared witness set, a protocol that achieves it with credentialed sessions, sequence-bound receipts and signed closing counts, and a proof (Theorem 1) that omission, fabrication, relabelling or substitution of a witnessed action requires a witness-key or orchestrator-key forgery.

2. **A separation between document verification and stateful reconciliation** (Theorem 2): a verifier whose verdict is a function of a single submitted document achieves neither witnessed completeness nor replay-freedom. We use it to place in-toto/SLSA/TUF, the agent-provenance drafts and receipt schemes on one side of the boundary and the gate on the other, and to explain the omission results.

3. **An evidence-bound, fail-closed release gate**: a bundle format compatible with in-toto v0.2 / SLSA v1.0 envelopes, a domain-separated, depth-bound Merkle trace attestation with truncation-resistant count binding, and a gate whose verdict is reproducible under a pinned policy version and holds replay state.

4. **A falsifiable evaluation with executed baselines and disclosed failures**: 7 omission attacks with an honest-execution control against executed in-toto/DSSE and TUF verifiers and re-implemented receipt and hash-chain designs; 17 tamper vectors, five of them authored against the design rather than the check list, with clean negative controls; a held-out inspection protocol; Wilson intervals on every rate; and an explicit account of the vector the gate does not block, the anomaly class inspection does not see, and the actions no witness covers.

Record formats versus authorization layers. in-toto and SLSA define *passive record formats*: they attest what occurred, and supply no stateful layer that gates a release on a non-deterministic trace. EviAssure is that layer, an active, fail-closed gate that holds replay state, binds the trace by a signed count and witness reconciliation, and decides under a pinned policy version, and Theorem 2 makes the distinction formal. Composition is preserved: an `EvidenceBundle` is an in-toto v0.2 / SLSA v1.0 envelope, so existing records ride inside it unmodified. Section 8.3 compares against the 2026 agent-provenance standards and receipt schemes.

Open Science and Reproducibility. The full implementation, the 1,075-profile execution trace corpus, the 17-vector attack harness with its clean negative controls, the omission suite, and the executed baselines (in-toto/DSSE, TUF, and the Open Policy Agent (OPA) Rego policy) are released as a sanitized anonymous review artifact (Appendix G).

2 Preliminaries

2.1 Formal Security Definitions

Definition 1 (Unforgeability under Chosen Message Attack (EUF-CMA))**.** An Evidence Assurance Scheme $\Sigma = (\mathrm{KeyGen}, \mathrm{Package}, \mathrm{Verify})$ is EUF-CMA secure if for all probabilistic polynomial-time adversaries $\mathcal{A}$, the probability that $\mathcal{A}$ outputs a valid pair (E^*, σ^*) such that $\mathrm{Verify}(PK, E^*, \sigma^*) = 1$ without E^* having been signed by

[1]The complete anonymous source code, evaluation corpus, and benchmark suite are available at https://anonymous.4open.science/r/eviassure-artifact-781D/.

an honest party holding SK is negligible in security parameter λ:

$$\Pr[\mathrm{Exp}^{\text{euf-cma}}_{\Sigma,\mathcal{A}}(\lambda) = 1] \leq \mathrm{negl}(\lambda) \tag{1}$$

Definition 2 (Merkle Trace Inclusion Soundness)**.** For root R_M committed together with its leaf count N, and leaf H_i, a Merkle proof π_i is sound if no computationally bounded adversary can construct π'_i for any $H'_i \neq H_i$ (including internal node digests of the same tree) such that $\mathrm{VerifyMerkle}(H'_i, \pi'_i, R_M, d) = 1$ for the committed depth $d = \lceil \log_2 N \rceil$, where verification accepts only proof paths of exactly length d and leaves and internal nodes are hashed under disjoint domain prefixes.

Definition 3 (Evidence Integrity)**.** An evidence pack (R_M, σ, N) comprising a Merkle root R_M, signature σ, and signed trace count N provides *evidence integrity* if no probabilistic polynomial-time adversary, given oracle access to the honest packager, can produce a verifying pack (R'_M, σ', N') that the packager never issued; equivalently, nothing that was *recorded* can be modified, reordered, inserted, or removed without detection. Integrity is a property of the signed record and is guaranteed by Theorem 3 below.

Definition 4 (Trace Completeness)**.** Let E be an agent execution and let $\mathcal{C}$ be the evidence collector that observes E. A recorded trace set $T = \{T_1, \ldots, T_N\}$ is *complete with respect to* E if every action of E that $\mathcal{C}$ observed is represented by exactly one T_i. Completeness is an observation property of the *collector*, not of the Merkle tree: the tree attests exactly the leaves it was given. If a real execution is $A \to B \to C \to D \to E$ but the recorded trace is $A \to B \to D \to E$, the tree protects the recorded sequence perfectly and says nothing about C. No signature, hash chain, or inclusion proof over the submitted set can, because the adversary forged nothing; it declined to create evidence. Definition 5 makes this attainable by moving the observation out of the collector's trust domain.

Definition 5 (Witnessed Trace Completeness)**.** Let $\mathcal{W} = \{W_1, \ldots, W_m\}$ be a set of *witnesses*, each holding a signing key the adversary does not control, and let $\mathsf{med}(W_j)$ be the set of action types W_j mediates. An action is *witnessed* if its type lies in $\bigcup_j \mathsf{med}(W_j)$. Let $\mathcal{O}$ be the *orchestrator*, the trusted controller that opens a release evaluation: for release *rid* it issues one *session credential* $\mathrm{Sign}_{\mathcal{O}}(sid, rid)$, and every witness serves a request only if it carries a credential that verifies under $\mathcal{O}$'s pinned key. A submitted trace set T for release *rid* is *witness-complete* if T declares the session *sid* that $\mathcal{O}$ credentialed for *rid*, and for every $W_j \in \mathcal{W}$ there is a W_j-signed closing (sid, n_j), the W_j-signed receipts in T are exactly the contiguous sequence $1, \ldots, n_j$ for *sid*, each receipt binds to a distinct member of T, and every witnessed action in T carries a receipt.

Completeness is stated *relative to* $\mathcal{W}$: an action of a type no witness mediates is outside the guarantee. This is a deployment-controllable condition (mediate every externally observable effect) rather than the unbounded assumption that the collector observed everything.

Theorem 1 (Witnessed Completeness Soundness)**.** Let $\mathcal{A}$ control the agent, the evidence collector and the collector's signing key, but no key in $\mathcal{W}$ and not $\mathcal{O}$'s key. If the gate accepts a witness-complete bundle for release *rid* with credentialed session *sid*, then except with probability $(m+1) \cdot \mathrm{Adv}^{euf\text{-}cma}_{\mathrm{Ed25519}}$, the trace set contains every witnessed action performed for *rid*, and every witnessed action it contains was performed.

Proof in Appendix C.

Theorem 2 (Record Formats Cannot Achieve These Properties)**.** Let V be a *stateless document verifier*: a deterministic predicate $V(d) \in \{\text{accept}, \text{reject}\}$ over a single submitted document d, with access to fixed public parameters (trust roots, policy) but no state that varies with prior submissions and no channel to $\mathcal{W}$. Then V achieves neither replay-freedom nor witnessed completeness.

Proof in Appendix C.

Theorem 2 is why we describe in-toto, SLSA, TUF, AAS-1, `agent-decision` and receipt schemes as record formats rather than as weaker gates. Their verification is by construction a function of the submitted document; the gap is structural, not a matter of maturity, and Section 7.6 measures it.

3 System Architecture and Threat Model

Intuition. EviAssure turns an agent's step-by-step execution path into a signed, Merkle-committed evidence bundle before deployment, has the components that served the agent's actions attest to how many they served, and evaluates the bundle at a *fail-closed gate* that denies release whenever a required proof, signature, receipt, closing or threshold is missing, malformed, or belongs to a different session.

3.1 System Model

The architecture involves six entities (Figure 1): the *Agent*, the *Evidence Collector*, the *Witnesses*, the *Orchestrator*, the *Registry*, and the *Release Assurance Gate*.

Agent and Evidence Collector: the agent runtime and the component that records its actions into an `EvidenceBundle` signed under the collector key. Both are in the adversary's tier. In production the collector is an out-of-process monitor outside the agent's sandbox, observing activity via eBPF syscall tracepoints and container network proxies; *in this evaluation it is Python instrumentation wrappers* (Section 7.10). Signing authority for the collector key is rooted outside the runner in a KMS/HSM; the gate checks the declared key ARN against `kms_key_arn_pattern`, a misconfiguration control rather than an adversarial one. **Witnesses**: the tool endpoints, API gateways and sandbox brokers that actually serve the

agent's actions. Each holds its own key, verifies the session credential before serving, issues a receipt per action and a closing per session (Section 4.2). They are a separate trust domain from the collector. **Orchestrator**: the trusted CI controller that opens a release evaluation by issuing one signed session credential per release and, at the end, tells the gate which session it credentialed. In the reference implementation the same process later runs the gate. **Registry**: an out-of-band store for release policies, the pinned collector keys, the pinned witness keys with the action types each mediates, the pinned orchestrator key, and the revocation list. **Release Assurance Gate**: the enforcement entity, running in a trusted CI/CD step, evaluating the signed `EvidenceBundle` against the pinned policy and reconciling it against the witnesses for the credentialed session.

3.2 Adversary Model

The security model considers a powerful adversary $\mathcal{A}$ attempting to deploy an unapproved, regression-prone, or malicious agent update to production environments [8, 28, 29]. $\mathcal{A}$ may control the agent, the evidence collector and the collector's signing key, intermediate runner environments, network transit layers, and build scripts.

Trust Model Boundaries. EviAssure assumes Ed25519 and SHA-256 remain computationally intractable for PPT adversaries, and that the Cloud KMS / HSM faithfully enforces identity access control over private signing keys. Witness keys and the orchestrator's credential key are distinct trust domains: the adversary holds neither, and Section 7.10 states what degrades when that assumption is weakened. It further requires strict environment isolation: an agent that escapes its sandboxed tier to obtain code execution on the gate, a witness or the orchestrator can forge attestations *prior* to signing. Compromise of the gate's trusted computing base, of a witness, or of the orchestrator is therefore an explicit out-of-scope limitation: the cryptographic guarantees hold assuming a tamper-proof control plane. HSM side channels, subversion of the provider's root KMS keys, and hypervisor rootkits are likewise out of scope.

We categorize $\mathcal{A}$'s in-scope capabilities into six classes. Five are mapped to the tamper vectors of Table 1: runner and memory tampering ($\mathcal{A}_1$), network and signature forgery ($\mathcal{A}_2$), replay and time skew ($\mathcal{A}_3$), governance bypass ($\mathcal{A}_4$), and serialization malleability ($\mathcal{A}_5$). The sixth, *collector omission* ($\mathcal{A}_6$), is the adversary of Theorem 1: it performs actions and declines to record them, relabels them into another session, or presents another session's bundle, and it is mapped to the omission vectors of Table 2.

3.3 Security Requirements

The construction is proved against three requirements, subject to the trust boundaries above; the evaluation in Section 7 is organised to test each in turn.

1. **Evidence Integrity**: no recorded action can be modified, reordered, inserted, or removed without detection (Definition 3).
2. **Trace Completeness**: the attestation must bind the count of recorded actions so that truncation is not silent (Definition 4); and, because a count the collector itself signs cannot constrain the collector, must be strengthened to hold *relative to a witness set* (Definition 5).
3. **Release Authorization Soundness**: the decision must be a deterministic function of signed evidence, the credentialed session and pinned policy. No PPT adversary should induce an APPROVED verdict for modified, incomplete or substituted traces.

The gate enforces these fail-closed: any payload failing verification is rejected with exit code 1 (`BLOCKED`). Requirement 2 is where every prior system stops at the *collection boundary* [1, 16]; Section 4.2 is our attempt to move it.

4 Cryptographic Evidence Attestation

EviAssure establishes verifiable release claims by packaging agent action histories into cryptographically attested evidence bundles. This section describes the Merkle tree aggregation protocol, browser Document Object Model (DOM) attestation, trace parameter blinding, and SLSA v1.0 / in-toto predicate envelopes.

4.1 Execution Trace & Browser UI Attestation Schema

Each execution trace record T_i captures structured action metadata across autonomous agent steps. For API and backend tool calls, T_i records:

$$T_i = (\mathrm{id}_i, \mathrm{agent}_i, \mathrm{action}_i, \mathrm{status}_i, \mathrm{duration}_i, H_{\mathrm{out},i}) \quad (2)$$

For browser and UI agent steps, EviAssure extends attestation to capture DOM state hashes (H_{DOM}) and visual screenshot render digests (H_{screen}):

$$T_{\mathrm{UI},i} = (\mathrm{id}_i, \mathrm{agent}_i, \mathrm{action}_i, \mathrm{url}_i, \mathrm{elem}_i, H_{\mathrm{DOM},i}, H_{\mathrm{screen},i}) \quad (3)$$

$H_{\mathrm{DOM},i}$ is computed over a deterministically serialized DOM, so identical page states yield identical digests and any mutation (an injected modal, an altered form control) changes

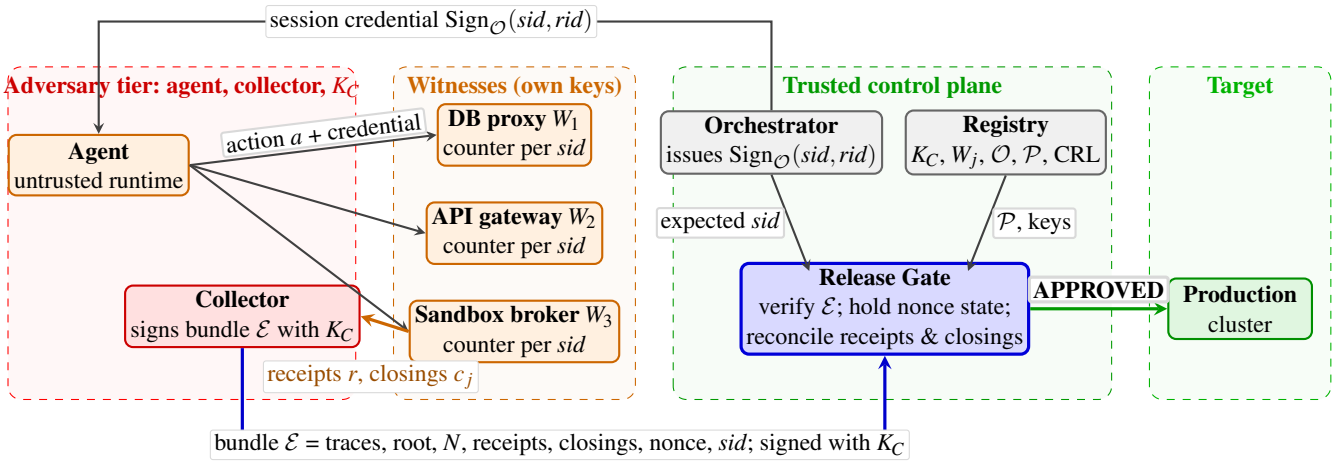

Figure 1: EviAssure with witnesses. The adversary controls the agent, the collector and the collector's key K_C, but no witness key and not the orchestrator's key. Witnesses serve only requests carrying the orchestrator's session credential, sign a receipt for every action they serve and a closing count at session end; the collector packages traces and attestations into a bundle signed under K_C; the gate verifies the bundle, holds replay state, and reconciles the trace set against receipts and closings *for the session the orchestrator credentialed*, so an omitted, fabricated, relabelled or substituted witnessed action requires a witness-key or orchestrator-key forgery. Actions outside every witness's mediated set are outside the guarantee.

it; $H_{\text{screen},i}$ digests canvas pixels under deterministic software rasterization (headless browser, GPU and font hinting disabled, pinned viewport), which removes the sub-pixel drift that would otherwise make an honest re-render fail its own digest. Each record yields a canonical leaf digest $H_i = \text{SHA256}(T_i.\texttt{to\_hash}())$ committing *every* schema field, so editing any recorded field after signing, including `duration_ms`, invalidates the recomputed root and is rejected by the gate.

4.2 Witness Protocol

A witness is the component that *actually serves* an agent action: a tool endpoint, an API gateway, a database proxy, a sandbox syscall broker. It is not part of the agent runtime, and the security argument rests on two properties: the adversary who owns the agent and the collector holds neither a witness key nor the orchestrator's credential key.

Sessions are opened by the orchestrator, not named by the agent. A release evaluation begins when the orchestrator issues a session credential $\text{Sign}_{\mathcal{O}}(sid, rid)$ for release rid and injects it into the agent's environment. Every witness pins $\mathcal{O}$'s public key and serves a request only if it carries a credential that verifies; a bare session string, or a credential signed by anyone else, is refused and the action is not performed. Without this rule the guarantee below would be empty: an adversary who owns the agent and the collector could label the actions it wants to hide with a different session identifier at the witness and submit a complete, honestly reconciling bundle for the benign one: omission without a forgery (vector O7, Section 7.6). With it, relabelling requires forging $\mathcal{O}$'s signature, and the gate binds each release to the one session $\mathcal{O}$ credentialed for it.

Each witness keeps a monotonic counter per session. On serving action a in session sid it returns

$$r = \text{Sign}_{W_j}(sid,\ \text{seq},\ H_{\text{act}}(a),\ \text{prev}) \tag{4}$$

where H_{act} digests the action identity the witness can bind a receipt to: in the implementation the trace identifier, agent identifier, action name and output digest; the collector-side `status` and `duration_ms` fields are deliberately not attested by the witness; and prev is the digest of the witness's previous receipt in the session, so the receipts also form a per-issuer hash chain and the chaining baseline of Section 7.6 can be evaluated on the same records. At session end, on the orchestrator's broadcast of the credential, W_j emits $\text{Sign}_{W_j}(sid, n_j)$, committing to how many actions it served. Every registered witness closes every session, including with $n_j = 0$: without that rule, dropping a witness's actions *and* its closing is indistinguishable from never using the tool, which is vector O3.

The gate reconciles, for the session the orchestrator credentialed: require the bundle to declare that session; authenticate every attestation under a registry-pinned witness key; require the session identifier to match; require each witness's receipts to form exactly $1..n_j$; require every receipt to bind to a submitted trace; and require every trace claiming a mediated action to carry a receipt. The bundle signature covers a digest of the whole attestation set, so a collector cannot delete receipts from a signed bundle and leave the signature intact.

Cost and deployment. A receipt is one Ed25519 signature over about 200 bytes at the point of service, and the gate performs $O(|T|)$ verifications. The real cost is organisational: witnesses must be deployed at the boundaries that matter, and Definition 5 makes the guarantee exactly as strong as that deployment.

Whether such a set can be assembled is the natural objection. A witness is a component that already exists (an API gateway, a database proxy, a sandbox broker), and the change is to check a credential and sign what it already logs; witnesses need no agreement with each other, so coverage grows incrementally and a partial deployment is sound over what it does mediate; and the guarantee degrades *visibly*, because an unmediated action type is outside it by definition (Section 7.9 reports that residue for a concrete tool set). A perfect-collector assumption cannot be audited at all; a guarantee relative to a declared witness set can.

Parameter blinding. Bundles carry digests, never raw payloads. A keyed HMAC mode (`blind_payload`) also re-keys those digests per deployment, which buys unlinkability across environments and resistance to dictionary recovery of low-entropy outputs: SHA256(`"TOKEN_VALIDATED"`) is recoverable by exhaustive search, the salted HMAC is not. It is deterministic, so it does not conceal repetition within one deployment. We record it as a hardening measure rather than a contribution.

4.3 Merkle Trace Aggregation and Sparse Proof Compression

To attest N execution traces without transmitting full raw logs to every gate request, EviAssure aggregates normalized trace digests into a SHA-256 Merkle tree [19,21]. Each trace record T_i maps to leaf digest H_i.

Raw trace logs are 221.75 KB at $N = 1{,}000$ and 220,704 KB at $N = 10^6$; EviAssure transmits the 32-byte root R_M and $O(\log N)$ inclusion proof paths π_i for audited steps.

Given proof path $\pi_i = \{(P_1, \text{pos}_1), \ldots, (P_K, \text{pos}_K)\}$, inclusion is verified recursively by setting $C_0 = H_i$ (the committed leaf digest; leaf digests are already SHA-256 strings and are *not* re-hashed at the proof boundary, matching `verify_merkle_proof`) and evaluating:

$$C_j = \begin{cases} \text{SHA256}(C_{j-1} \parallel \text{":"} \parallel P_j), & \text{if pos}_j = \text{right} \\ \text{SHA256}(P_j \parallel \text{":"} \parallel C_{j-1}), & \text{if pos}_j = \text{left} \end{cases} \tag{5}$$

Leaves and internal nodes are hashed under *disjoint domain prefixes* following RFC 6962 [19]: a leaf commits SHA256(`0x00` $\parallel$ T_i.`to_hash()`) and an internal node commits SHA256(`0x01` $\parallel$ $L \parallel R$) over the raw 32-byte child digests. Hashing both as unprefixed SHA-256 over delimited ASCII, and arguing that a fixed node-preimage length supplies domain separation, does not work: the two input languages merely fail to overlap for one leaf schema, which is an accident of the schema rather than a property of the construction. Explicit prefixes make the separation structural.

The proof is valid if and only if $C_K = R_M$ *and* the path length equals the committed tree depth $d = \lceil \log_2 N \rceil$ derived from the signed execution trace count. The two defences are independent and both are necessary: domain separation prevents an internal digest from ever being interpreted as a leaf, and the depth bound prevents the shortened-path substitution of the CVE-2012-2459 family, which succeeds without any hash collision. Vector V15 (Section 7.4) exercises exactly this attack against the third-party auditor path. Transmission size for an audited step remains < 5 KB.

4.4 SLSA v1.0 and in-toto Predicate Envelopes

To integrate with cloud-native artifact registries and software supply chain attestors, EviAssure formats each generated `EvidenceBundle` into an in-toto v0.2 Statement envelope compatible with SLSA Provenance v1.0 specifications [30, 32]. The envelope carries `_type` `https://in-toto.io/Statement/v0.1`, the artifact digest as `subject`, `predicateType` `https://slsa.dev/provenance/v1`, and the `evidence_id` under `predicate.buildDefinition.externalParameters`; a complete example is in the artifact.

5 Fail-Closed Policy Verification Engine

Release authorization in EviAssure is governed by a zero-trust policy engine (`ReleasePolicyEngine`) that enforces mandatory fail-closed default rules. This section details the declarative policy schema, multi-factor evaluation protocol, key governance rules, and the system-level formal security analysis.

5.1 Declarative Policy Governance Rules

A release policy specification P defines strict quantitative safety thresholds and cryptographic verification conditions:

$$P = (P_{\text{min\_pass}}, P_{\text{max\_drift}}, P_{\text{min\_sigs}}, P_{\text{valid\_window}}, \mathcal{K}_{\text{trusted}}, \mathcal{K}_{\text{rev}}, \mathcal{A}_{\text{KMS}}) \tag{6}$$

where:

- $P_{\text{min\_pass}}$: Minimum required unit and integration test pass rate (default: 100.0% (100/100)).
- $P_{\text{max\_drift}}$: Maximum allowable unresolved security drift findings (default: 0).
- $P_{\text{min\_sigs}}$: Required threshold number of valid signatures from authorized keys (default: $K = 1$).

- $P_{\text{valid\_window}}$: Evidence timestamp freshness validity window $\Delta T_{\max} = 3,600$ s with future clock-skew tolerance $\delta_{\text{skew}} = 30$ s (assuming RFC 5905 NTP synchronisation).
- $\mathcal{K}_{\text{trusted}}$: Trusted key registry pinning authorized key IDs to Ed25519 public keys (`governance/trusted_keys.yaml`). Verification uses the pinned key, never one supplied inside a bundle; unregistered IDs are rejected. Authority thus rests on a small, versioned, offline-pinnable root set, following key transparency [20] and delegation hierarchies [17].
- $\mathcal{K}_{\text{rev}}$: Key Revocation List (CRL) tracking compromised or deprecated key identifiers.
- $\mathcal{A}_{\text{KMS}}$: Required Cloud KMS or HSM key ARN boundary pattern matching.

Listing 2 outlines the complete zero-trust verification algorithm enforced by `ReleasePolicyEngine`.

5.2 Formal Security Analysis

The analysis has two layers. Theorem 3 is primitive-level: a game sequence reducing to EUF-CMA and collision resistance. Theorem 4 is system-level: what an APPROVED verdict *means*, and the trust boundaries (collector, key registry, nonce state) that meaning rests on.

Theorem 3 (Fail-Closed Release Soundness)**.** Under the assumption that Ed25519 [3] is EUF-CMA secure [6, 11] and SHA-256 is collision-resistant, no probabilistic polynomial-time adversary $\mathcal{A}$ can induce `ReleasePolicyEngine` to output APPROVED for a modified trace history, forged signature, replayed nonce, or non-compliant evidence payload except with negligible probability $\text{negl}(\lambda)$.

Proof in Appendix C.

Theorem 3 bounds an adversary's ability to *fake* evidence; it says nothing on its own about what a genuine, honestly-signed evidence pack entitles its holder to. That is the release-gate question, and it is Theorem 4.

Theorem 4 (Evidence-Bound Release Authorization Soundness)**.** Fix a policy version P and a trusted key registry $\mathcal{K}_{\text{trusted}}$. If `ReleasePolicyEngine` outputs APPROVED for evidence E, then, except with probability $\text{negl}(\lambda)$, all of the following hold:

(i) **Authenticity:** E carries a valid Ed25519 signature by a non-revoked key pinned in $\mathcal{K}_{\text{trusted}}$; attacker-generated keys, spoofed key IDs, and revoked keys cannot produce a valid signature.

(ii) **Evidence binding:** the recomputed Merkle root over the submitted trace set equals the signed root, the signed trace count equals the number of submitted traces, and every inclusion proof binds to the committed tree depth under disjoint leaf/node domains. The trace set the gate evaluated is therefore *the one the collector signed*: no trace was added, removed, reordered or edited between collection and adjudication. This says nothing about the relation between the signed trace set and the underlying execution; that gap is Definition 4 and is not closed by any cryptographic check.

(iii) **Freshness:** the timestamp lies within the validity window $\Delta T_{\max}$ and the skew bound δ_{skew}, and the nonce has not been processed within the window; the release is not a replay of an earlier approved bundle.

(iv) **Policy compliance:** all declared thresholds (test pass rate, unresolved drift, KMS ARN boundary, signature threshold K) satisfy policy version P; an APPROVED verdict under a different policy version is not an APPROVED verdict for this release.

Scope and Operational Assumptions. The guarantees hold conditionally on (a) the *collector assumption* of Definition 4 (the evidence collector observed and recorded every action of the release execution), which supplies the completeness half of the guarantee, and (b) the *key-management assumption*: registry keys are honestly generated and the HSM/KMS enforces the ARN boundary. The theorem asserts nothing about actions that never reached the collector, nor about the semantic correctness of recorded trace content; both are defined in the threat model and handled by the out-of-band semantic inspection layer.
Proof in Appendix C.

6 Deployment Integration and Workload Specimens

EviAssure runs as a GitHub Action against two synthetic workload specimens: a backend multi-step agent (`specimens/agent_runner.py`: JWT claims, vector retrieval, sandboxed SQL) and a browser agent (`specimens/web_app_runner.py`: navigation, clicks, DOM and screenshot digests), both simulations, and a third (`specimens/live_agent_runner.py`) that drives a real tool-calling LLM through witnessed tools (Section 7.9). The production collector of Section 3 (eBPF syscall mediation) remains an architectural proposal rather than an evaluated component, which is why Section 4.2 moves the completeness guarantee onto witnesses a compromised collector cannot speak for.

6.1 CI/CD Integration

The gate ships as a GitHub Action that loads the pinned key and witness registries and the policy, in-

gests the `EvidenceBundle` together with the orchestrator's release request naming the credentialed session, runs `ReleasePolicyEngine` (`scripts/verify_release_gate.py`), and on failure emits structured diagnostics and exits non-zero on every output path. The shipped policy requires witnessed completeness: an unwitnessed bundle, or one whose session is not the credentialed one, is BLOCKED, and the shipped registry (`governance/witness_registry.yaml`, written by `scripts/provision_witnesses.py`) carries the demo witnesses and orchestrator key so that the packaging script and the CLI exercise the full witnessed path end to end; the CLI-level omission tests run against exactly that configuration.

6.2 Agent Execution Trace Corpus ($N = 1,075$)

The corpus (`corpus/agent_trace_corpus.json`) contains 1,075 profiles / 5,450 trace records across five agent architectures, four to six steps each: 1,000 clean, 50 *overt* anomalies over ten classes, and 25 *stealth* anomalies over five classes. It is entirely synthetic and templated, as its `provenance` field states (Section 7.10).

The overt/stealth split guards against a benchmark that grades itself. If every planted anomaly used an action name and status absent from clean traffic and the detector keyed off a list of exactly those names, generator and detector would be one list and the reported recall a property of the generator. *Overt* classes have that structure (out-of-vocabulary action and status) and are the easy half. *Stealth* classes are the point: a mutating statement issued through the read-only query path, a ledger transfer to an attacker-controlled destination, retrieval returning injected context, extraction used as an exfiltration channel, a linter invoked with findings suppressed. Each uses an action, a status and a duration drawn from clean traffic, so no per-leaf vocabulary or envelope test can separate them. Section 7.8 reports both halves separately.

7 Systems Security Evaluation

The empirical evaluation follows a deliberate two-part strategy: a micro-level validation of the gate's enforcement mechanisms, and a macro-level evaluation of the system deployed at corpus scale.

Mechanism evaluation. Baseline performance is measured through Merkle scaling latency and multi-core verifier throughput. Adversarial resilience is measured against the vector suite: twelve vectors derived from the five capability classes ($\mathcal{A}_1$–$\mathcal{A}_5$), each exercising one enforcement, plus five authored against the design rather than against the check list: the ones with the power to fail, and three of which found defects during development. Clean negative controls measure the false-block rate, without which a block rate is uninterpretable. The bundles on this axis carry no witness attestations by construction, so the gate is evaluated here with witness reconciliation disabled (the *unwitnessed evaluation profile*, `witnessed=False`, recorded in the artifact); the omission suite (Section 7.6) tests reconciliation itself, the axis the tamper vectors structurally cannot reach, and the shipped policy requires it.

System-level deployment. A corpus-scale evaluation over 1,075 agent trace profiles exercises the interaction between the cryptographic integrity gate and an out-of-band semantic inspection layer, and shows where that interaction still leaves a gap.

Results are split by kind. *Deterministic* security results (block counts, ablation, inspection) are produced by `scripts/run_security_eval.py` into `results/security_evaluation.json` and reproduce identically on any machine. *Timed* measurements are wall-clock timings recorded by `scripts/run_release_benchmark.py` into `results/benchmark_summary.json` on an Apple M4 Max (14 cores, no SMT) running Python 3.14.x; each is the mean over 5 runs, with the standard deviation recorded alongside in the artifact. Verdicts are deterministic checks evaluated once, so run-to-run dispersion is zero by construction; sampling uncertainty over the finite vector set is reported instead, as a Wilson 95% interval on every rate.

7.1 Merkle Scaling up to $N = 1,000,000$ Traces

Merkle build time was evaluated from $N = 10$ to $N = 10^6$ over 5 runs per point (Figure 2). Construction scales to 10^6 traces in 1770.01 ms (mean; $\sigma = 18.36$ ms across runs), and to 175.5 ms for $N = 100,000$. We do not report a "packaging overhead" ratio: post-tree bundle construction and signing cost a constant independent of N, and dividing a constant by an assumed per-step execution time that grows with N produces a curve that falls for arithmetic reasons rather than measuring any property of the system. Merkle construction, the component that actually scales, is what is reported.

7.2 Parallel Verifier Throughput

Verifier throughput (Figure 3) was evaluated using a Python `ProcessPoolExecutor` across 1, 2, 4, 8, and 16 worker cores over 1,000 evaluation requests per run and 5 runs per configuration; each evaluation is a distinct fresh-nonce bundle against a warm policy engine (evidence creation and replay-state checks are measured separately, so the timed region is steady-state verification). Throughput rises from 3,657 ops/s on a single worker to a peak of $7{,}025 \pm 50$ operations/second (mean $\pm$ std) at 4 workers (a $1.9\times$ speedup), then declines; each task is a three-trace bundle shipped to a worker process, so the measurement is dominated by inter-process serialization rather than verification, and we do not attribute the decline to hardware. The figure characterises *small* bundles: because the gate recomputes the whole tree (Listing 2), per-bundle

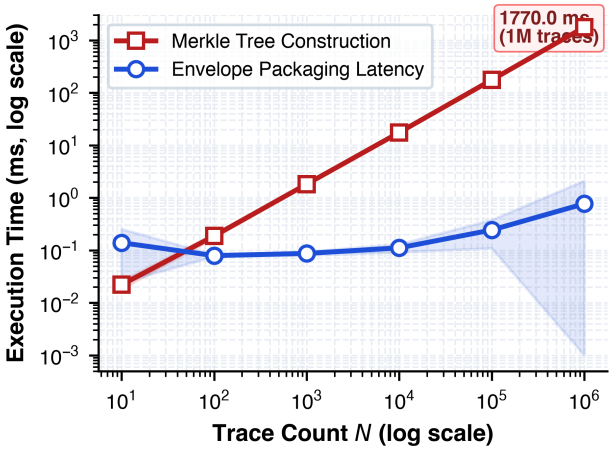

Figure 2: Merkle construction and envelope packaging from $N = 10$ to $N = 10^6$, mean over 5 runs with the standard deviation as a band. Both axes are logarithmic. The callout is read from `results/benchmark_summary.json` rather than typed, so it cannot diverge from 1770.01 ms in the text.

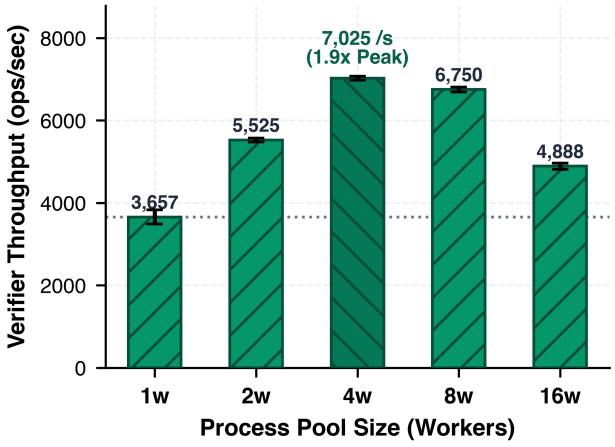

Figure 3: Verifier throughput against process-pool size, mean ± standard deviation over 5 runs of 1,000 evaluations. The peak bar and its ratio are selected from the data, not fixed to a worker count. Throughput is measured on three-trace bundles and does not describe large-N verification.

verification cost is linear in N, and at $N = 10^6$ the gate is bounded by the 1770.01 ms rebuild, not by this number.

7.3 Sparse Inclusion Proofs

Auditors verifying individual high-risk steps need not reprocess full logs (220,704 KB at $N = 10^6$): at that size the committed depth is $d = 20$, proof generation takes 0.003 ms, and the 1.93 KB payload of 20 node digests verifies in 0.010 ms.

7.4 Adversarial Vector Suite

EviAssure blocks 16 of 17 scored vectors (94.1%, 95% CI [73.0, 99.0]), and falsely blocks 0 of 4 clean negative controls. Three points about how that number should be read.

The mechanism vectors do not demonstrate resilience. V1–V12 map close to one-to-one onto the gate's checks, because we wrote both; they are a completeness test for the implementation and nothing more, and V14 and V17 (neither reachable from the check list) show why a 1:1 map proves nothing about the design.

Three vectors (†) found real defects during development. V14: a gate that falls back to the key carried *inside the bundle* when the registry is disabled approves an adversary's own keypair with a conformant ARN. V17: a length heuristic lets a 64-character leaf string be adopted as its own digest. V15: the CVE-2012-2459-family internal-node substitution against the auditor path. Each was found by writing the vector and is now blocked and regression-tested, which is the argument for authoring vectors against the design.

V16 is not blocked, and we report it. The same approved bundle presented to a second gate replica that does not share nonce state is APPROVED, because replay protection is a property of gate *state*, not of the evidence. The shipped verifier persists nonces and fails closed when the ledger is unavailable, but cross-replica correctness needs shared storage the prototype does not provide.

Differential wire fuzzing. A fuzzing campaign whose every mutation perturbs a field inside the signed payload cannot fail (a verifying bundle is unreachable and no clean case exists), so it carries no information. Ours perturbs the *serialization* of a validly signed bundle (key order, insignificant whitespace, unicode escaping, number formatting, duplicate keys, byte-order marks) where both outcomes are reachable. Over 1000 cases (20% clean controls): 217/217 semantics-changing encodings blocked, 783/783 semantics-preserving encodings approved. The campaign found a real defect during development: a duplicate-key detector that accumulated key counts across every object in the document registered sibling objects in the `traces` array as duplicates, so any deployment supplying the raw wire text rejected every multi-trace bundle. It is fixed and regression-tested.

7.5 Per-Check Ablation

Each enforcement was disabled in turn and the suite re-evaluated (matrix in the artifact's `ablation` block). Disabling the signature check releases V1, V3, V9, V14 and V18; removing Merkle re-derivation releases V2 and V8; the quality, drift, freshness and replay checks each release exactly the vectors they target (V7; V6; V5 and V11; V4). V16 escapes every variant, including the full gate.

Table 1: Adversarial vectors and gate outcomes (unwitnessed evaluation profile). V1–V12 target one enforcement each; V14–V18 were authored against the design, independently of the check list; † marks the three that found a defect during development. V13 is retained as a configuration check but is not scored (Section 7.10).

ID	Vector	Class	Outcome
Mechanism vectors (one per enforcement)			
V1	Unsigned payload	$\mathcal{A}_2$	Blocked
V2	Tampered trace digest	$\mathcal{A}_1$	Blocked
V3	Attacker-key signature	$\mathcal{A}_2$	Blocked
V4	Replayed nonce (same gate)	$\mathcal{A}_3$	Blocked
V5	Expired timestamp	$\mathcal{A}_3$	Blocked
V6	Unresolved drift	policy	Blocked
V7	Sub-threshold pass rate	policy	Blocked
V8	Forged Merkle root	$\mathcal{A}_1$	Blocked
V9	Revoked key ID	$\mathcal{A}_4$	Blocked
V10	Duplicate-key wire doc	$\mathcal{A}_5$	Blocked
V11	Future clock skew	$\mathcal{A}_3$	Blocked
V12	Trace-set truncation	$\mathcal{A}_1$	Blocked
Independent vectors (authored against the design)			
V14	Attacker key, registry off	$\mathcal{A}_2$	Blocked†
V15	Internal node as leaf	$\mathcal{A}_1$	Blocked†
V16	Cross-replica nonce replay	$\mathcal{A}_3$	**Not blocked**
V17	64-char leaf confusion	$\mathcal{A}_1$	Blocked†
V18	Post-signing DOM mutation	$\mathcal{A}_1$	Blocked
Scored total		16/17 (94.1%)	
Clean negative controls		0/4 falsely blocked	

Two results are worth stating precisely. First, not every check is necessary against this suite: disabling the signed trace-count binding releases no vector, because V12 is independently caught by root recomputation; the count binding earns its place by protecting the *auditor* path, whose proof depth is derived from the signed count. Second, emptying the trusted-key registry collapses the gate rather than weakening it: all Ed25519 evidence is rejected, which is only true of an implementation that never falls back to a bundle-supplied key; V14 checks it.

7.6 Omission Attacks

The tamper suite and the omission suite test different things. A tamper vector modifies evidence and is caught by an integrity check. An omission vector modifies *nothing*: the adversary performs an action and declines to record it, or performs it in a session the gate never sees, then signs honestly over what it presents. There is no discrepancy to detect, which is why Theorem 2 rules the class out for document verifiers rather than merely predicting they will do badly at it.

We construct seven (Table 2) plus an honest fully-witnessed control that every system must approve, without which a reconciler that rejects everything would score perfectly. O1–O4 truncate: an interior action, the trailing action, every action of one witness, and the trailing action with a forged lower closing. O5 splices receipts from a second credentialed session into the gap. O6 fabricates a mediated action no witness served. O7 performs the dangerous action in one credentialed session and submits the complete, honestly witnessed bundle of a second, benign credentialed session for the release. Every session is opened by an orchestrator credential; a request that carries none, or a credential signed by anyone else, is refused by the witness before any bundle exists, and that refusal is exercised by a test rather than scored as a vector.

Table 2: Omission attacks detected, out of 7. All systems approve the honest control (OC1). in-toto/DSSE and TUF are executed verifiers; the receipt and chaining verifiers are re-implemented from their published designs and run on the same records (Section 4.2); every row is executed, none is a presence check.

	Verification approach	Detected
in-toto/DSSE	signature over the document	0/7
TUF	expiry, thresholds, revocation	0/7
AAS-1 style	per-issuer hash chaining	2/7
Receipts (2026)	per-action receiver attestation [10, 25]	2/7
EviAssure	full gate, no reconciliation	0/7
EviAssure	**+ witnessed completeness**	**7/7**

Three results are worth reading carefully. First, EviAssure without reconciliation detects 0 of 7 despite enforcing signatures, Merkle re-derivation, count binding, freshness and replay. The mechanism, not some incidental check, is what closes the class.

Second, per-action receipts detect 2 of 7. Receipts catch the two attacks that leave a trace *without* a matching attestation (cross-session splicing and fabrication) and miss all four truncations, because dropping an action drops its receipt and the remaining pairs still match; they also miss substitution, because the substituted bundle's receipts are perfectly matched to its own session. This is the empirical form of the paper's central observation: a receipt proves an action happened; a set of receipts does not prove no other action happened, nor that it happened in the session under evaluation.

Third, hash chaining detects 2 of 7. A chain links records that are present, so an interior gap breaks it and a spliced foreign receipt fails to link, but a suffix truncation does not (the end simply moves), and neither does dropping a whole issuer, forging a closing the chain never consults, fabricating an unchained trace, or substituting a session whose chain is intact. This is the precise limitation of AAS-1's omission story, and it is why the closing count and the credentialed session, not the chain, are what make truncation and substitution detectable.

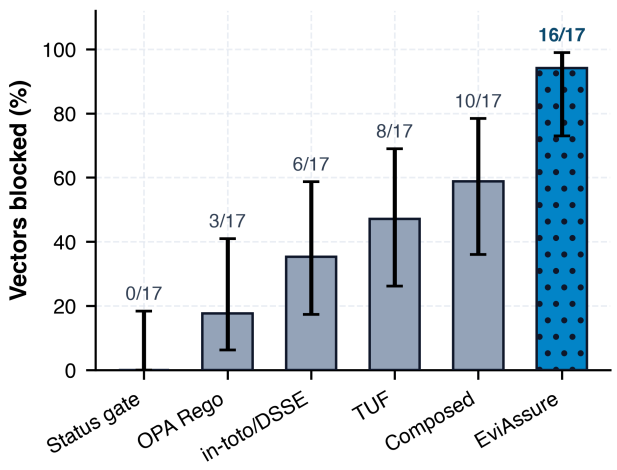

Figure 4: Executed baselines against the 17-vector suite, with Wilson 95% intervals. The intervals overlap, and that is the point: at this sample size the evaluation supports a qualitative account of *which* vectors survive a modern composed pipeline, not a strong quantitative separation (Section 7.7).

7.7 Comparative Gate Analysis

Each baseline is configured as a competent engineer would deploy it, and executed; counts and intervals are in Figure 4. in-toto/DSSE (DSSE v1 pre-authentication encoding + Ed25519 verification) authenticates the envelope against a *pinned trust root* rather than a key travelling with the payload. TUF evaluates Timestamp-role expiry with `python-tuf`'s metadata objects over that trust-root check, so it supplies `expires`, a signature threshold and root-driven revocation. OPA Rego (`governance/baseline_opa.rego`) is executed by the `opa` binary; the evaluation script's `-require-executed` flag, and the artifact packager, refuse to record or ship a modelled substitute for any of these three. The composed pipeline is DSSE $\wedge$ TUF $\wedge$ OPA. The unauthenticated status gate is retained as a *floor*, not a competitor.

The honest headline is therefore 16/17 versus 10/17 (Figure 4), and the intervals overlap: with 17 vectors this evaluation cannot support a strong quantitative separation, and we do not claim one. What it does support is a *qualitative* account of which vectors survive a modern composed pipeline and why. The composed baseline blocks every authentication, freshness, revocation and policy-threshold vector; TUF alone blocks 8, since freshness, revocation and threshold signing were solved in 2010 [26].

The vectors it does not block are the ones this paper is actually about: V2 and V8 (trace-level integrity: no baseline recomputes a Merkle root over an action history), V12 (trace-set truncation), V15 and V17 (proof-path and leaf-typing attacks on the audit surface), and V4 (within-gate replay, which requires the verifier to hold state across submissions rather than to validate a document). That is the delta: *binding the release decision to the execution trace, and holding state while deciding*. It is a narrow claim, and it is one the evaluation can carry.

Table 3: Layered evaluation over 1,075 profiles / 5,450 trace records. L3 uses a detector fitted on clean profiles only.

Layer	Check	Result	Rate
L1 Integrity	Merkle root consistency	1,075/1,075	100%
L1 Integrity	Depth-bound proofs valid	1,075/1,075	100%
L2 Gate	Clean APPROVED	1,000/1,000	100%
L2 Gate	Anomalous APPROVED	75/75	100%
L3 Inspect	Overt recall	50/50	100%
L3 Inspect	**Stealth recall**	**0/25**	**0%**
L3 Inspect	Overall recall	50/75	66.7%
L3 Inspect	Clean falsely flagged	0/1000	0%

7.8 Corpus-Scale Layered Evaluation

Neither the tamper nor the omission suite reaches the semantic axis: a complete, witnessed, correctly signed bundle can still record behaviour nobody authorised. The corpus measures that third layer (Table 3).

L1 and L2 are as before: all bundles are Merkle-consistent with valid depth-bound proofs, all clean profiles are APPROVED, and *all* anomalous profiles are also APPROVED. We report the latter rather than hiding it: semantic misbehaviour that is cryptographically well-formed is outside the gate's scope by design.

L3 is a held-out protocol. The detector consults no list of anomalous actions. It is fitted on clean profiles alone, learning an action vocabulary, a status vocabulary and a per-action duration envelope, and every anomaly class is held out from that fit. Under this protocol it recalls 50/50 overt anomalies and 0/25 stealth anomalies, overall 50/75 (66.7%, CI [55.4, 76.3]) at 0/1000 false positives.

The zero is the result. Per-leaf inspection detects agents that do things they have never done before; it is blind to an agent doing a permitted thing for an impermissible reason, which is precisely what a prompt-injected agent looks like in a trace. Combined with L2, the layered picture is: the gate proves what was recorded is genuine, inspection catches structural novelty, and *neither* catches semantically disguised misbehaviour.

7.9 Live Agent Sessions

Everything above is computed over a templated corpus, which leaves the paper's own premise untested: does witnessed completeness survive an agent that actually decides what to do? We therefore ran 31 sessions of real tool-calling LLMs (5 model families, 4 providers) against a small tool set fronted by witnesses of Section 4.2. The model chooses the actions;

Table 4: Live agent sessions (5 model families, 4 providers). Coverage is the share of the agent's calls that went to the four witnessed tools; the fifth tool is unwitnessed by design. It is split by how the session ended, because a run stopped by the turn cap never reaches the unwitnessed call.

	Ran to completion	**Truncated**
Sessions	15	16
Witness coverage	**82.7%**	100.0%
Over all 31 sessions		
Distinct action sequences	22/31 (71%)	
Honest control reconciles	31/31	
Omission attacks detected	177/177 (6 vectors)	

nothing is scripted (Table 4). What the harness does and does not separate must be stated before its numbers are read. Each witness runs outside the collector: it executes the tool, digests the output *it* produced, and signs with a key never exported from its own address space, so an altered or dropped record no longer matches its receipt. Separation is by process, key and credential (and by container, with no network and a read-only repository, in 18 sessions) on one host: not by host or trust domain, nor against root or a debugger.

The sequences are genuinely non-deterministic , 22 of 31 sessions were distinct (71%), over 6.45 ± 2.234 actions, and the agent generated its own failures, malformed SQL refused by the read-only path (`DENIED`) and queries against non-existent tables (`ERROR`), rather than executing a happy path we wrote for it.

Reconciliation is sound on real traces: the honest control reconciles 31/31, and re-deriving the omission attacks against each live session detects 177/177. That total covers 6 of the 7 vectors: O5 need a second session (a concurrent one for the splice, a second credentialed one for the substitution) and were not constructed for the recorded runs. A vector that cannot be constructed for a given session (an interior omission in a three-action run, say) is dropped rather than scored, since counting an attack that never ran as detected is how a suite flatters itself. Detection is cryptographic, so this confirms rather than surprises; but a protocol that failed here would have been refuted, and it was not.

Witness coverage is a property of the tool set, and it was 82.7%. Four of the five tools are witnessed and `emit_summary` is not, so coverage is the share of the agent's calls that went to the four; the pooled figure over all 31 sessions (91.6%) overstates it, because only 15 sessions ran to completion and a session stopped by the turn cap never reaches the unwitnessed call, giving 100.0% by construction of the cap. The number reports the residue Definition 5 leaves open for this deployment (roughly one action in five of a completed session lay outside the guarantee), and its value is that it is declared and measurable rather than assumed away, not that it is a discovery about agents. 11 of 42 sessions were lost to provider quota, credit and rate limits, which we report rather than retry into silence.

7.10 Limitations and Threats to Validity

The corpus is synthetic and the live harness is small. Section 7.9 runs a real tool-calling LLM against the witness protocol, so the completeness claim is not evaluated only on templates. The 1,075-profile corpus behind Section 7.8 and the semantic-inspection results is synthetic and templated, as the artifact's own `provenance` field declares. The live sessions are modest in scale (31, of which 15 ran to completion), 4-provider, use benign tools, and separate witness from collector only as far as Section 7.9 states; they establish that the protocol composes with a real agent, not how it behaves under an adversarial one, and their coverage figure describes the tool set rather than agents in general.

The collector is unimplemented. Every security guarantee is conditioned on complete mediation at the collection boundary (Definition 4). The production design grounds this at the OS boundary via eBPF; the evaluation uses Python instrumentation wrappers. The load-bearing component is therefore an architectural proposal, not a measured one.

The gate does not block semantic misbehaviour, and inspection misses the hard half. L2 approves all 75 anomalous profiles by design; held-out L3 inspection recalls 0/25 stealth anomalies (Section 7.8). Content-level policy over trace semantics is future work.

V16 is unblocked. Replay protection is a property of gate state. The shipped verifier persists nonces and fails closed when the ledger is unavailable, but cross-replica correctness requires shared storage (Redis/DynamoDB) that the prototype does not implement.

Baseline scope. DSSE/in-toto and TUF are executed verifiers and OPA is executed by its binary; the receipt and hash-chain baselines of Section 7.6 are our re-implementations of published designs, not their reference code, and are labelled as such in the artifact; the unauthenticated status gate is modeled and reported as a floor, not a competitor.

Witness and orchestrator trust, and availability. Witnessed completeness is exactly as strong as the declared witness set. An adversary holding a witness key mints valid receipts and closings for that witness; one holding the orchestrator's key mints credentials and can relabel or substitute sessions; and action types no witness mediates lie outside the guarantee by construction (Definition 5). We evaluate neither witness or orchestrator compromise nor key rotation, and a witness that fails to close a session blocks the release: fail-closed, but a denial-of-service surface a deployment must plan for. The gate must be told the credentialed session by the orchestrator, out of band; a deployment that lets the bundle nominate its own session has no defence against O7.

Vector-set construct validity. V1–V12 were authored alongside the checks they exercise and demonstrate only that

those checks fire. V14–V18 were authored against the design and three of them found defects during development, which is evidence that the suite has some power, but a 17-vector suite gives wide intervals (CI [73.0, 99.0]), the 7-vector omission suite mirrors the case analysis of Theorem 1, and no suite authored by the system's designers can establish resilience to novel attack.

Measurement and key management. Timed figures are means over 5 runs on one platform. The artifact signs with static demo keys so results reproduce; there is no real KMS round trip, so no KMS latency is measured or claimed.

Ethics. The work raises no human-subjects concerns; the ethical and dual-use analysis, including responsible disclosure, is Appendix D.

8 Related Work

Positioning. Threshold signing, metadata expiry, rollback protection and key revocation are *not* novel here: TUF has supplied them since 2010 [26], which is why our executed TUF baseline blocks 8 of 17 vectors (Section 7.7). EviAssure's claim is narrow: it binds the release decision to a Merkle-attested *execution trace* for a credentialed session, reconciles it against a second trust domain, and holds replay state, which none of the systems below does.

8.1 Supply Chain Provenance

in-toto [32], SLSA [30] and Sigstore/Cosign [23] provide cryptographic provenance for build pipelines and image digests, including vetted runtime-trace predicates [14]; any release pipeline should use them for the static artifact side. What they do not provide is a stateful authorization layer: freshness and replay are not integral to step attestations, and release decisions are not bound to attested traces under a pinned policy version, and EviAssure supplies that layer while remaining envelope-compatible.

8.2 Policy Engines and Workload Identity

Policy engines (OPA [27], Kyverno [31]) evaluate declarative rules over admission payloads but compute no Merkle root over an action history and hold no replay state; workload identity frameworks (SPIFFE/SPIRE [13], Macaroons [4]) establish *who* the runner is, not what it did. Update frameworks are the closest prior art: TUF [26] and its delegation extensions [17, 22] supply expiry, thresholds, rollback and revocation, which is why we execute TUF as a baseline rather than claim those properties.

8.3 AI-Agent Provenance Standards and Receipts

Three 2026 initiatives address agent provenance as a *record format*. The in-toto `agent-decision/v0.1` RFC [2] standardises the shape of a signed policy-decision record, leaving freshness, replay and revocation to the verifier. APAS [1] builds provenance chains on in-toto Statements and grades itself candidly: at its lower levels provenance is "self-attested — an attacker who compromises the orchestrator can forge records." AAS-1 [16] defines evidentiary record classes with per-issuer hash chains that detect omission *between* records a verifier already holds, which, as Section 7.6 measures, a suffix truncation evades.

The closest designs are receiver-attested receipts. Notarized Agents [10] has the service that serves an agent call sign a receipt binding the call's input and output digests, encrypt it to the agent's owner and publish it to a transparency log, and names what it does not solve: the "suppression attack", in which an agent that does not call a service produces no receipt, and set-completeness of the receipts a log returns. PipeLab's action receipts [25] are signed by a mediator outside the agent process and hash-chained. Both give per-receipt verifiability from a second trust domain, which is the same move as ours; neither gives a verifier a way to know that the receipt set is *all* of them. How completeness could instead be obtained from a transparency log, and why the closing count does not need one, is Appendix A.

Theorem 2 explains why record formats and receipt sets alone are structurally short of completeness: each verifies a submitted document. Their records ride inside EviAssure's envelope unmodified; what it adds is the state, the second trust domain and the session binding a document cannot carry.

9 Conclusion

EviAssure binds a release decision to a domain-separated, depth-bound Merkle attestation of an agent's execution trace, evaluated by a stateful fail-closed gate. Its central claim is Witnessed Trace Completeness: reconciling collector evidence against signed witness receipts and closing counts, for the session the orchestrator credentialed, detects 7/7 omission attacks every record-format baseline misses, a class Theorem 2 shows no stateless verifier can close. On the tamper axis it blocks 16/17 against 10 for a composed in-toto/DSSE + TUF + OPA pipeline and reports the one it fails; held-out inspection recalls 50/50 overt and 0/25 disguised anomalies. Evidence-bound release assurance makes an unauthorised release undeniable and attributable, not impossible.

References

[1] Agentic Research. Agent provenance attestation standard (apas). Draft v0.2.1, reference implementation

"rosary" / `signet`, 2026. Draft standard for cryptographically verifiable provenance chains across autonomous AI-agent pipelines, from work decomposition through agent execution to code delivery; uses in-toto Statement envelopes and DSSE signing.

[2] Auxidus Technologies. Rfc: agent-decision/v0.1 predicate for ai agent policy decisions. in-toto/attestation issue #554, in-toto Attestation Framework (CNCF), 2026. Open RFC proposing an in-toto attachment predicate for AI agent authorization decisions: agent identity, principal, policy evaluations, allow/deny outcomes, hashed tool arguments, timestamps, and trace correlation.

[3] Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012.

[4] Arnar Birgisson, Joe Gibbs Politz, Úlfar Erlingsson, Ankur Taly, Michael Vrable, and Mark Lentczner. Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud. In *NDSS Symposium*, 2014.

[5] Henk Birkholz, Antoine Delignat-Lavaud, Cédric Fournet, Yogesh Deshpande, and Steve Lasker. An architecture for trustworthy and transparent digital supply chains. RFC 9943, IETF (Proposed Standard), 2026.

[6] Jacqueline Brendel, Cas Cremers, Dennis Jackson, and Mang Zhao. The provable security of ed25519: Theory and practice. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1659–1676, 2021.

[7] Justin Cappos, Justin Samuel, Scott Baker, and John H Hartman. A look in the mirror: Attacks on package managers. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 565–574, 2008.

[8] Nicholas Carlini, Matthew Jagielski, Christopher A Choquette-Choo, Daniel Paleka, Will Pearce, Hyrum Anderson, Andreas Terzis, Kurt Thomas, and Florian Tramèr. Poisoning web-scale training datasets is practical. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, 2024.

[9] Ruian Duan, Omar Alrawi, Ranjita Pai Kasturi, Ryan Elder, Brendan Saltaformaggio, and Wenke Lee. Towards measuring supply chain attacks on package managers for interpreted languages. In *NDSS Symposium*, 2021.

[10] Juan Figuera. Notarized agents: Receiver-attested confidential receipts for ai agent actions. arXiv preprint arXiv:2606.04193, 2026.

[11] Shafi Goldwasser, Silvio Micali, and Ronald L Rivest. A digital signature scheme secure against adaptive chosen-message attacks. *SIAM Journal on Computing*, 17(2):281–308, 1988.

[12] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *ACM Workshop on Artificial Intelligence and Security (AISec)*, pages 79–90, 2023.

[13] CNCF Identity Working Group. Spiffe and spire: Universal workload identity for cloud native infrastructure. In *Cloud Native Computing Foundation (CNCF) Technical Report*, 2020.

[14] in-toto Attestation Framework contributors (CNCF). Runtime trace predicate, in-toto attestation framework. Predicate specification, https://github.com/in-toto/attestation/blob/main/spec/predicates/runtime-trace.md, 2024.

[15] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[16] Kadikoy Limited. Agent auditability standard (aas-1), working paper v0.1. Draft for public comment, companion to the AIS-1 agent identity standard, 2026. Open standard for audit-grade evidentiary records of autonomous agent activity: five record classes (Action, Batch, Continuous, Determination, Engagement), twelve audit assertions, JCS canonicalization, SHA-256 hashing, and per-issuer hash chains for omission detection.

[17] Trishank Karthik Kuppusamy, Santiago Torres-Arias, Vladimir Diaz, and Justin Cappos. Diplomat: Using delegations to protect community repositories. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2016.

[18] Piergiorgio Ladisa, Henrik Plate, Matías Martínez, and Olivier Barais. Sok: Taxonomy of attacks on open-source software supply chains. In *IEEE Symposium on Security and Privacy (IEEE S&P)*, pages 1509–1526, 2023.

[19] Ben Laurie, Adam Langley, and Emilia Kasper. Certificate transparency. In *RFC 6962, IETF*, 2013.

[20] Marcela S Melara, Aaron Blankstein, Joseph Bonneau, Edward W Felten, and Michael J Freedman. Coniks: Bringing key transparency to end users. In *USENIX Security Symposium*, pages 383–398, 2015.

[21] Ralph C Merkle. A certified digital signature. *Advances in Cryptology—CRYPTO'89 Proceedings*, pages 218–238, 1989.

[22] Marina Moore, Trishank Karthik Kuppusamy, and Justin Cappos. Artemis: Defanging software supply chain attacks in multi-repository update systems. In *Annual Computer Security Applications Conference (ACSAC)*, pages 1002–1017, 2023.

[23] Zachary Newman, John Speed Meyers, and Santiago Torres-Arias. Sigstore: Software signing for everybody. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 2353–2367, 2022.

[24] Chinenye Okafor, Taylor R Schorlemmer, Santiago Torres-Arias, and James C Davis. Sok: Analysis of software supply chain security by establishing secure design properties. In *ACM Workshop on Software Supply Chain Offensive Research and Ecosystem Defenses (SCORED, co-located with ACM CCS)*, pages 15–24, 2022.

[25] PipeLab. Agent action receipts: Signed AI audit evidence. Technical note, 21 May 2026, 2026. Ed25519 receipts signed by a mediator outside the agent process, hash-chained to the previous receipt, verifiable offline.

[26] Justin Samuel, Nick Mathewson, Justin Cappos, and Roger Dingledine. Survivable key compromise in software update systems. In *ACM Conference on Computer and Communications Security (ACM CCS)*, pages 177–190, 2010.

[27] Torin Sandall and Tim Hinrichs. Policy-based access control with open policy agent. Conference presentations and project documentation, 2018. https://www.openpolicyagent.org/.

[28] Roei Schuster, Congzheng Song, Eran Tromer, and Vitaly Shmatikov. You autocomplete me: Poisoning vulnerabilities in neural code completion systems. In *USENIX Security Symposium*, pages 1559–1575, 2021.

[29] Adam Shostack. *Threat Modeling: Designing for Security*. John Wiley & Sons, Indianapolis, IN, 2014.

[30] Google Open Source Security Team. Supply-chain levels for software artifacts (slsa) framework specification. In *Linux Foundation OpenSSF Technical Report*, 2023.

[31] Nirmata Engineering Team. Kyverno: Kubernetes native policy management engine. In *Cloud Native Computing Foundation (CNCF)*, 2022.

[32] Santiago Torres-Arias, Anish Kathpal, Hiten Gupta, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *USENIX Security Symposium*, pages 1393–1410, 2019.

[33] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Use of Generative AI and Automated Tooling

The authors disclose the use of AI coding assistants for manuscript drafting assistance, LaTeX typography checking, test-harness refactoring, and an automated build-and-consistency pipeline that regenerates every numeral in this paper from the artifact's results files. All references were verified by the authors against primary sources (publisher records, arXiv, or the live specification pages). The core conceptual architecture, security proofs, empirical findings, and intellectual claims remain the original work of the authors, who take responsibility for all content.

A Extended Related Work

Publishing receipts to a transparency service in the SCITT model [5] or a Certificate-Transparency-style log [19] is one route to that: a verifier that audits the log for every receipt in a session obtains completeness relative to the log, at the cost of a log operator in the trust domain and a query whose own completeness must be authenticated. The closing count is the other: each witness commits, under its own key, to how many actions it served, so completeness relative to the witness set needs no log and no cross-witness agreement, and the credentialed session pins which set the gate reconciles. The two compose, since receipts and closings can be logged, and we regard the log as the natural place to keep closings for post-hoc audit.

B Algorithms and Policy Listings

The body refers to these by number; they are reproduced here so the main text stays within the page limit.

```
LEAF, NODE = b"\\x00", b"\\x01"   # RFC 6962 domain
    separation

def leaf_digest(content):
    return sha256(LEAF + content.encode()).hexdigest()

def node_digest(l, r):
    return sha256(NODE + bytes.fromhex(l)
                       + bytes.fromhex(r)).hexdigest()

def build_merkle_tree(leaf_digests):
    current, levels = leaf_digests, [leaf_digests]
    while len(current) > 1:
        current = [node_digest(current[i],
                    current[i+1] if i+1 < len(current)
                                 else current[i])
                   for i in range(0, len(current), 2)]
        levels.append(current)
    return current[0], levels
```

Listing 1: Merkle Trace Aggregation Protocol.

```
# In: Evidence E, Policy P, revoked K_rev, seen N_seen
# Out: D in {APPROVED, BLOCKED}; any exception => BLOCKED
def evaluate_release_gate(E, P, K_rev, N_seen):
    if N_seen is None: return BLOCKED  # no replay state
    valid = set()
    for s in E.signatures:
        if s.key_id in K_rev: continue
        if s.key_id not in K_trusted: continue   #
            registry
        pk = K_trusted[s.key_id]                 # pinned
            only:
        if s.pub_key and s.pub_key != pk: continue
        if not match_kms_arn(s.kms_arn, P.kms): continue
        if verify_ed25519(canon(E), s.sig, pk):
            valid.add(s.key_id)          # distinct
                signers
    if len(valid) < P.min_signatures:     return BLOCKED
    if merkle_root(E.traces) != E.merkle_root: return
        BLOCKED
    if E.traces_count != len(E.traces):   return BLOCKED
    if E.test_pass_pct < P.min_pass_pct:  return BLOCKED
    if E.unresolved_drift > P.max_drift:  return BLOCKED
    if E.nonce in N_seen:                 return BLOCKED
    if not fresh(E.timestamp, P.window, skew=30): return
        BLOCKED
    N_seen.add(E.nonce);  return APPROVED
```

Listing 2: Fail-Closed Policy Gate Verification Protocol.

C Deferred Proofs

The theorem statements, their assumptions and the intuition behind each appear in the body; the full arguments are reproduced here so the main text stays within the page limit.

Proof of Theorem 1. A witnessed action performed for *rid* is served by some W_j under a credential; since $\mathcal{O}$ issues one credential per release, that credential names *sid* unless $\mathcal{A}$ minted one, an Ed25519 forgery under $\mathcal{O}$'s key. So suppose $\mathcal{A}$ omits a witnessed action a served by W_j under *sid* as its k-th action, and the gate accepts. W_j's counter is monotone and advances only on service, so W_j issued receipts $1,\dots,n_j$ with $k \le n_j$, and its closing commits to n_j. Acceptance requires the receipt sequence for W_j to be exactly $1,\dots,n'_j$ with a matching closing. Three cases. (i) $n'_j = n_j$: then $\mathcal{A}$ supplied n_j distinct receipts, but only $n_j - 1$ correspond to submitted actions, so some receipt is unmatched and reconciliation rejects, unless $\mathcal{A}$ minted a receipt, an Ed25519 forgery under W_j. (ii) $n'_j < n_j$: the presented closing differs from the one W_j signed, so $\mathcal{A}$ forged a closing under W_j. (iii) $\mathcal{A}$ substitutes a receipt from another session $sid' \neq sid$, or presents a complete bundle for sid': the session identifier is inside the signed receipt and closing payloads and the gate reconciles only against the *sid* it credentialed for *rid*, so both are rejected without forgery. Each surviving case is a forgery against one of m witness keys or the orchestrator key; a union bound gives the stated advantage. Fabrication of a witnessed action is symmetric: the action carries no receipt, and minting one is again a forgery. □ □

Proof of Theorem 2. Replay-freedom. Determinism gives $V(d) = V(d)$: if V accepts d once it accepts every resubmission, so an adversary replays an approved release indefinitely. Distinguishing first from subsequent submission requires state that varies across submissions, which V by definition lacks.

Witnessed completeness. Fix a witness W and consider two executions: E_1 serving n actions, and E_2 serving the same first $n-1$ and then a further action whose record the adversary omits. Let d_1, d_2 be the honestly-signed documents. Every attestation in d_2 is genuine: the collector signs truthfully over what it recorded, and each retained receipt is authentic. So d_1 and d_2 are identically distributed over every field V can inspect, and $V(d_1) = V(d_2)$. Since V must accept d_1 (a correct execution), it accepts d_2. The only field that could differ is a commitment to W's served count, which W alone can produce, and requiring it makes the verifier's input depend on a trust domain outside d, contradicting statelessness. □

Proof of Theorem 3. Two hops carry content. $\mathbf{Game}_1$ aborts if $\mathcal{A}$ produces a valid signature over a payload the honest packager never issued; an EUF-CMA adversary $\mathcal{B}$ installs its challenge key as the sole registry entry and answers packaging requests through its signing oracle on $\mathrm{canon}(E)$, so the simulation is perfect and a Game_1 abort is a forgery on a never-queried message. $\mathbf{Game}_2$ aborts on $\mathcal{T}^* \neq \mathcal{T}$ with equal root; under domain-separated leaves and the committed depth this needs a SHA-256 collision. Replay, freshness, revocation, registry membership and threshold comparison are table lookups over the signed payload, rejecting with probability 1 given correct gate state, and contribute no term. Hence $\mathrm{Adv}^{\mathrm{tamper}} \le \mathrm{Adv}^{euf\text{-}cma}_{\mathrm{Ed25519}}(\mathcal{B},t,q_s) + \mathrm{Adv}^{coll}_{\mathrm{SHA256}}(\mathcal{C},t)$, read at the 128-bit level rather than asymptotically in λ, since both primitives are fixed-parameter. □

Proof of Theorem 4. **(i)** Follows from Theorem 3, Game_1, plus the deterministic registry checks enforced before any signature verification (`policy.py`): unknown key IDs, bundle-supplied keys disagreeing with the registry, and revoked keys are rejected outright, so per-signature success is bounded by $\mathrm{Adv}^{\text{euf-cma}}_{\mathrm{Ed25519}}(\mathcal{A})$. **(ii)** Follows from Theorem 3, Game_2, plus the deterministic signed-count check $N = |\text{traces}|$; substitution requires either a hash collision or a broken signature. **(iii)** Freshness, skew, and nonce checks are deterministic (Game_3; Listing 2) and hold with probability 1 given correct gate state, with the disclosed caveat that replay protection requires sharing the nonce cache across gate replicas in distributed environments. **(iv)** Threshold comparisons are deterministic over the signed payload and the pinned policy version. Composition yields the statement; the only probabilistic terms reduce to the primitive advantages of Theorem 3. □

D Ethics and Dual-Use Analysis

The research presented in this paper introduces a defensive mechanism for the security and provenance of autonomous agent releases. This work does not involve human subjects,

does not analyze personally identifiable information, and does not conduct active measurements against production systems or networks operated by third parties. All datasets, trace profiles and harnesses are synthetic and locally generated, and no live credentials were used at any point. The live-agent sessions of Section 7.9 call commercial model APIs under the authors' own accounts, with synthetic tools over a local in-memory database; no third-party system is probed and no data leaves the harness beyond the prompts themselves. The 17 adversarial tamper vectors and the 1,075-profile corpus are fully synthetic and generated within isolated, local test environments for the sole purpose of evaluating the proposed cryptographic and policy enforcement mechanisms.

Dual-Use Considerations. Cryptographic execution attestation mechanisms, if repurposed maliciously, could theoretically be employed to construct invasive employee monitoring frameworks or enforce rigid software lock-in. EviAssure mitigates these dual-use concerns by enforcing decentralized multi-signature threshold governance and open policy schemas (`governance/release_policy.yaml`), and by keeping raw payloads out of evidence bundles entirely: traces carry digests only. Salted HMAC blinding (`blind_payload`) also unlinks the same recorded action across deployments, though as Section 4.2 states, it does not conceal payload equality within one deployment.

Data Privacy & Parameter Blinding. Autonomous software agents frequently process confidential prompt texts, customer financial details, or proprietary source code snippets. To prevent public release audit logs from exposing sensitive data, EviAssure keeps raw payloads out of evidence bundles entirely (traces carry digests only), and the blinding protocol also re-keys those digests with a per-deployment salted HMAC (H_{blinded}), so the same recorded action is not linkable across environments. Within one environment the deterministic construction still reveals repetition (Section 4.2). This preserves full Merkle tree inclusion auditability without storing proprietary inputs in the audit log.

Responsible Disclosure & Safety Impact. EviAssure raises the cost of forging or replaying release evidence produced by a compromised runner. It does *not* prevent prompt injection: an injected agent produces genuine actions that the collector honestly records and the gate correctly approves, and Section 7.8 measures this directly: all 75 anomalous corpus profiles, including the injection class, are APPROVED. The contribution is that such a release becomes *undeniable and attributable* after the fact, not that it is prevented. An earlier version of this appendix claimed prevention; that claim contradicted our own Section 7.8 results and has been withdrawn.

E Forensics and Diagnostics

For incident response after a release, an offline forensic utility (`scripts/audit_trace_forensics.py`) reconstructs the Merkle tree from a serialized `EvidenceBundle`, verifies signatures against the pinned registry and revocation list, and isolates leaf-index anomalies. Presented with a tampered payload it reports the exact leaf position i and the output-hash mismatch, emitting structured JSON for security operations centre (SOC) ingestion.

F Deployment Trade-offs

Two engineering consequences follow. *Audit granularity*: since a depth-bound proof costs 1.93 KB against 220,704 KB of raw trace, organisations can archive payloads to cold storage and keep only proofs online. *Replay state*: the gate must hold state across submissions, which V16 shows it cannot do across replicas without shared storage; a key-value store with atomic set-if-absent and TTL $\Delta T_{\max}$ is the natural implementation, and it must fail closed when that store is unreachable rather than admit a split-brain replay. Neither deployment is measured here.

G Open Science Appendix

The reproducibility materials consist of the complete EviAssure implementation (cryptographic attestation, blinding, held-out inspection, and policy verification modules), the 1,075-profile agent execution trace corpus, the 17-vector tamper harness with negative controls, the executed DSSE/TUF/OPA baselines, production GitHub Action workflows, benchmark scripts, and the paper figure generator. All datasets used to establish comparative baselines (e.g., the specific OPA Rego policies and JSON payloads used to evaluate the baseline) are included to guarantee full baseline reproducibility.

G.1 Anonymous Review Artifact & Verification

To comply with USENIX Security '27 Open Science guidelines while preserving double-blind review, the complete artifact has been published to a sanitized, anonymous repository at:

`https://anonymous.4open.science/r/eviassure-artifact-781D/`

The same content is also archived as `eviassure_usenix27_artifact.zip` (460.2 KB), built by `scripts/prepare_anonymous_artifact.py` with fixed member timestamps and sorted entries, so the archive is a deterministic function of the source rather than of the machine that packed it. Its SHA-256 is:

8544facb56e0f4515370a5a78d03528b
be8e4307a104f3f0768843b419f65935

We state plainly what this digest does and does not give a reviewer. It fixes the archive we retain, and it lets a reader who obtains that archive confirm they hold the same bytes. It is *not* independently checkable from the anonymous repository above: the archive is not served there, and the packager's forbidden-term list is deliberately withheld from the artifact (it is a list of author-identifying strings), so the build cannot be reproduced from the mirror alone. The reviewer-verifiable claims are the ones in the reproduction workflow below: the test suite, and the `results/` files every reported number is generated from.

G.2 Reproduction Workflow

To execute full reproduction of paper figures, benchmark results, and unit tests:

1. **Environment Setup**: Install dependencies via `pip install -e ".[dev]"` (Python $\geq$ 3.10); the `dev` extra supplies `pytest` and `hypothesis`, which the test suite imports.

2. **Unit & Integration Test Suite**: Run `pytest tests/ -v` (120 tests). The suite includes property-based crypto soundness tests, trusted-key-registry tests, the omission suite with its honest control, the credential-refusal and session-substitution tests, and an end-to-end test that drives `scripts/verify_release_gate.py` against the shipped registry and policy; `test_docs_consistency.py` also binds the manuscript's stated test count to the collected count, so the two cannot drift.

3. **Attestation Scaling & Throughput Microbenchmarks**: Run `python3 scripts/run_release_benchmark.py -repeats 5` to generate `results/benchmark_summary.json`.

4. **Adversarial & Baseline Evaluation**: Run `python3 scripts/run_security_eval.py --require-executed` (with `python-tuf` installed and the `opa` binary on `PATH`) to execute the 17-vector tamper suite, the clean negative controls, the wire-fuzzing campaign, the 7-vector omission suite with its honest control, the executed DSSE/TUF/OPA baselines, the ablation matrix and the held-out inspection protocol into `results/security_evaluation.json`, then `python3 scripts/write_security_macros.py`. These results are platform-independent. `--require-executed` fails rather than silently substituting a modeled baseline, and the artifact packager refuses to ship a results file that records one.

5. **Corpus Layered Evaluation**: Run `python3 scripts/generate_trace_corpus.py` then `python3 scripts/run_corpus_eval.py` to rebuild the 1,075-profile corpus and evaluate it through the integrity, policy and held-out inspection layers.

6. **Figure Generation & Manuscript PDF Rebuilding**: Run `python3 scripts/generate_paper_pdf.py` to regenerate paper plots and recompile `docs/usenix_paper_manuscript.pdf`.